ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



HỌC PHẦN: AN TOÀN PHẦN MỀM

ĐỀ TÀI: FUZZING WHITE BOX TESTING CHO QUẢN LÝ DÒNG HỌ

Giảng viên: ThS. Vũ Quang Dũng

Nhóm 5:

1. Nguyễn Tiến Lực (23010862 – K17 – KTPM(EL_2))

Lớp học phần: CSE703093-1-1-26(N02)

Hà Nội, tháng 10 năm 2026

DANH SÁCH PHÂN CÔNG

STT	Mã sinh viên	Họ và tên	Công việc
1	23010862	Nguyễn Tiến Lực	Phần 1 (Giới thiệu, bối cảnh, mục tiêu) Phần 2.2, 2.3 (Kiến trúc, luồng dữ liệu, chức năng trọng tâm). Phần 2.1, 2.4 (Chi tiết thành phần kho mã nguồn, quy mô) Phần 4.1 (Kết quả chạy 11 ca kiểm thử gốc) Phần 5.4 (Đề xuất khắc phục lỗi code và hoàn thiện bộ kiểm thử).

1. Giới thiệu và mục tiêu

Link github dự án: https://github.com/lucnguyencoder/software_safety

1.1. Bối cảnh và lý do lựa chọn đề tài

Hệ thống quản lý dòng họ lưu trữ thông tin cá nhân, quan hệ gia đình, sự kiện và các khoản thu chi của dòng họ. Dữ liệu gia phả có quan hệ phụ thuộc giữa các thành viên; vì vậy, một thao tác cập nhật sai có thể ảnh hưởng đến nhiều nút trong cây. Đồng thời, việc tổ chức dữ liệu theo từng dòng họ đặt ra yêu cầu kiểm soát quyền truy cập đến từng bản ghi, bên cạnh việc kiểm tra tài khoản đã đăng nhập.

Kho mã nguồn software_safety-main-2 gồm ứng dụng web quản lý dòng họ và chương trình C clan_genealogy_fuzzer.c. Chương trình C xây dựng một module trung tâm gồm năm bước xử lý, 16 trạng thái, 16 nhãn mệnh đề, 10 đường đi được khai báo và 11 ca kiểm thử định sẵn. Đây là cơ sở để nghiên cứu cách thiết kế ca kiểm thử theo trạng thái, theo dõi độ phủ và đánh giá tính đúng đắn của cơ chế phát hiện lỗi.

Một vấn đề cần giải quyết là sự khác nhau giữa trạng thái được chương trình đặt tên là lỗi và lỗi thực tế được công cụ độc lập xác nhận. Nếu chỉ dựa vào dòng chữ “CRASH DETECTED” do chính chương trình in ra, người kiểm thử có thể kết luận sai về tính an toàn của mã nguồn. Do đó, báo cáo kết hợp đọc mã nguồn, thực thi mô hình và kiểm tra bằng AddressSanitizer để đánh giá bằng chứng ở từng mức.

1.2. Mục tiêu cụ thể

Báo cáo hướng đến các mục tiêu sau:

Mô tả kiến trúc, luồng dữ liệu, bề mặt tấn công và phạm vi kiểm thử của hệ thống.

Phân tích một module xử lý năm bước cùng quan hệ giữa trạng thái, điều kiện kiểm tra, vùng nhớ và đường đi thực thi.

Chạy lại 11 ca kiểm thử có sẵn; xác định độ phủ trạng thái, mệnh đề và đường đi; kiểm chứng ý nghĩa của các nhãn “CRASH”, “HANG”.

Bổ sung ca kiểm thử biên để phát hiện sai sót mà bộ kiểm thử ban đầu chưa xét đến.

Nhận diện một số vấn đề ở mã nguồn ứng dụng web, phân biệt kết luận phân tích tĩnh với kết quả kiểm thử tích hợp.

Đề xuất hướng khắc phục và cung cấp thông tin cần thiết để tái lập thực nghiệm.

1.3. Phạm vi và mức độ kết luận

Phần thực nghiệm động tập trung vào chương trình C và các hàm bên trong chương trình này. Phần ứng dụng web tập trung vào đường đi dữ liệu của chức năng thành viên gia phả, middleware xác thực và mã khởi tạo cơ sở dữ liệu. Báo cáo không khẳng định đã kiểm thử toàn bộ ứng dụng, không thực hiện tấn công vào hệ thống đang triển khai và không đánh giá hiệu năng với tải lớn.

Các kết quả được phân thành ba nhóm: kết quả thực thi đã quan sát, vấn đề suy ra từ mã nguồn và đề xuất chưa triển khai. Chương trình C là mô hình độc lập, không gọi trực tiếp API Express hay cơ sở dữ liệu MySQL. Vì vậy, lỗi trong mô hình C không tự động trở thành lỗi của ứng dụng web.

1.4. Tóm tắt kết quả chính

Bộ 11 ca có sẵn đạt 16/16 nhãn trạng thái, 16/16 nhãn mệnh đề và 10/10 đường đi khai báo. Việc đối chiếu độc lập toàn bộ 11 trace trong log với PATH_TABLE cho thấy các chuỗi khớp với đặc tả tương ứng. Chương trình tự ghi tám nhãn “CRASH” và một nhãn “HANG”, nhưng kết thúc bình thường với mã 0 khi chạy cùng ASan/UBSan. Ca bổ sung có độ dài audit bằng -1 vẫn khiến ASan báo negative-size-param và dừng tiến trình.

Cấu trúc mới cho phép kết thúc sớm một ca khi gặp nhánh bất thường, nên trace phản ánh rõ vị trí dừng. Tuy nhiên, hàm ghi nhận path chỉ so khớp trạng thái cuối; tỷ lệ 100% chưa phải bằng chứng rằng mọi đường đi chương trình hoặc mọi miền đầu vào đã được kiểm thử. Báo cáo phân tích riêng kết quả quan sát và giới hạn của bộ đo này.

2. Mô tả hệ thống và chương trình mục tiêu

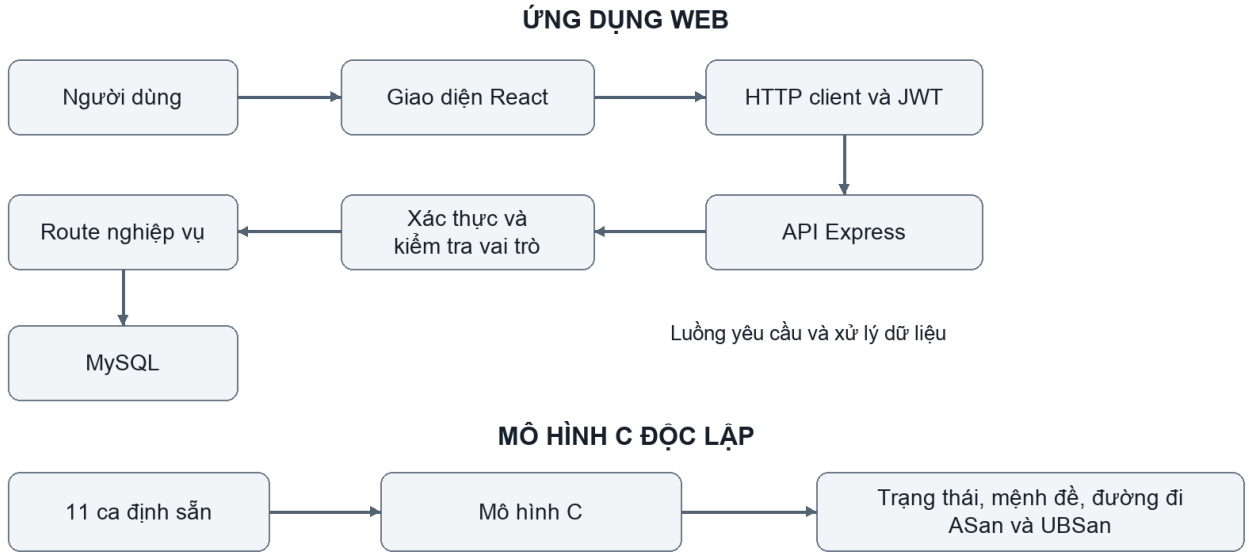
2.1. Các thành phần của kho mã nguồn

Thành phần	Công nghệ hoặc tệp chính	Vai trò
Giao diện web	React, TypeScript, Vite; quanly-dongho/src	Quản lý thành viên, hiển thị gia phả, sự kiện, tài khoản và tài chính

HTTP client	src/services/api.ts	Gửi yêu cầu JSON, đính kèm JWT trong header Authorization
Backend	Node.js, Express; backend/src	Xác thực, phân quyền, xử lý nghiệp vụ và truy vấn dữ liệu
Cơ sở dữ liệu	MySQL; backend/src/db	Lưu thành viên, tài khoản, sự kiện, giao dịch và nhật ký
Mô hình kiểm thử	clan_genealogy_fuzzer.c	Chạy các tình huống mô phỏng theo trạng thái và ghi log
Tài liệu đi kèm	README và bảng XLSX bốn sheet	Mô tả dự kiến về quy trình và kết quả; cần đối chiếu lại với code

Các phiên bản được khai báo trong package.json gồm React ^19.0.1, Vite ^6.2.3, Express backend ^4.19.2 và mysql2 ^3.10.1. Đây là khoảng phiên bản khai báo, không phải xác nhận phiên bản thư viện đã được cài đặt trong một hệ thống đang chạy.

2.2. Kiến trúc và luồng dữ liệu



Hình 2.2. Kiến trúc tổng quan. Nhánh ứng dụng web và nhánh mô hình C được thực thi độc lập. Liên hệ giữa hai nhánh nằm ở ý tưởng nghiệp vụ quản lý gia phả, chưa phải liên kết kiểm thử trực tiếp.

Ví dụ, khi người dùng cập nhật thành viên, giao diện gọi HTTP client, client gắn token rồi gửi yêu cầu đến API. Backend kiểm tra chữ ký token, trạng thái tài khoản và vai trò theo route. Dữ liệu được chuyển thành truy vấn SQL; kết quả trả lại dưới dạng JSON. Trong mô hình C, đầu vào được dựng sẵn trong cấu trúc FuzzTestCase, đi qua các hàm mô phỏng và không được lấy từ yêu cầu HTTP.

2.3. Chức năng và dữ liệu trọng tâm

Đối tượng thành viên có các trường họ tên, giới tính, tình trạng còn sống, ngày sinh, thế hệ, cha, mẹ và phối ngẫu. Backend cung cấp các thao tác liệt kê, xem chi tiết, thêm, sửa và xóa thông qua /api/members. Các bảng khác phục vụ tài khoản, sự kiện, phản hồi tham dự, giao dịch quỹ và nhật ký thao tác.

Với mô hình C, mỗi ca kiểm thử có năm nhóm dữ liệu: tên và tiểu sử; chỉ số thành viên và thao tác vòng đời; chỉ số cha, phối ngẫu và cờ tạo chu trình; số bản ghi quỹ; độ dài sao chép nhật ký. Đầu ra trên stdout gồm log chi tiết, chuỗi trạng thái của từng ca, ba tỷ lệ độ phủ và các bộ đếm do chương trình tự tăng. Code hiện không tự mở tệp log; cần chuyển hướng stdout hoặc dùng công cụ chạy lại để lưu bằng chứng.

2.4. Quy mô mã nguồn

Số liệu được đếm theo số dòng vật lý, bao gồm dòng trống và chú thích, không phải số dòng lệnh thực thi.

Phạm vi	Số tệp	Số dòng
clan_genealogy_fuzzer.c	1	532
quanly-dongho/src với đuôi mã nguồn được thống kê	28	15.189
quanly-dongho/backend/src	14	2.639
Tổng các tệp .c, .js, .ts, .tsx, .css, .html trong kho	45	18.395

Tổng toàn kho còn gồm các tệp ở ngoài hai thư mục src, nên không bằng tổng ba hàng đầu. Tệp C có 11 hàm được định nghĩa. Hàm execute_clan_genealogy_module là điểm thực thi nghiệp vụ trung tâm; hai hàm duyệt xử lý đồ thị, các hàm còn lại phục vụ log, đo độ phủ, tạo chiến dịch và

điều phối. Module_Clan_Genealogy_Engine là tên của một module logic, không phải năm module như phiên bản trước.

4. Kết quả và phân tích

4.1. Kết quả chạy 11 ca kiểm thử gốc

Bảng dưới đây ghi kết quả thực thi của bản cập nhật. Các tỷ lệ State/Pred trùng nhau vì chương trình đánh dấu hai loại nhãn tại cùng lời gọi enter_state.

Ca	Đầu vào hoặc thay đổi chính	Trạng thái mới	State/Pred	Path
TC01	Seed gốc; tên 13 byte; audit 12	S00, S03, S04, S10, S13, S15	37,5%	10%; PATH_01
TC02	Seed nhánh; slot 1; audit 18	Không	37,5%	10%; PATH_01
TC03	Tên dài 44 byte	S01	43,8%	20%; PATH_02
TC04	Tiểu sử chứa %x, %n	S02	50,0%	30%; PATH_03
TC05	parent_idx = -5	S08	56,2%	40%; PATH_04
TC06	spouse_idx = 15	S09	62,5%	50%; PATH_05
TC07	fund_count = 16385	S12	68,8%	60%; PATH_06
TC08	audit_claim_len = 60	S14	75,0%	70%; PATH_07
TC09	lifecycle_op = 1	S05, S06	87,5%	80%; PATH_08
TC10	lifecycle_op = 2	S07	93,8%	90%; PATH_09
TC11	create_cycle = 1	S11	100,0%	100%; PATH_10

Bảng 4.1. Bảng kết quả của chiến dịch. Cột Path ghi tỷ lệ tích lũy và mã path của ca hiện tại. Tỷ lệ được giữ theo cách làm tròn một chữ số thập phân của log; 9/16 được in là 56,2% trên môi trường thực nghiệm.

Nguồn: results/baseline.stdout.txt.

```
Total Test Cases Executed: 11 (2 Seeds + 6 BFS + 3 DFS)
State Coverage:     100.0% (16 / 16 States)
Predicate Coverage: 100.0% (16 / 16 Predicates)
Path Coverage:      100.0% (10 / 10 Execution Paths)
Bo dem CRASH do chuong trinh tang: 8
Bo dem HANG do chuong trinh tang:  1
Ma thoat thuc te cua baseline:    0
Chan doan ASan/UBSan trong stderr: khong co
```

Tên dài của TC03 là Nguyen_Van_Truong_Toc_Dai_Chi_Stack_Overflow, có strlen = 44 byte; vùng lưu chuỗi gồm ký tự kết thúc cần ít nhất 45 byte. Hai seed đều đi đến PATH_01; chín biến thể mở thêm chín lá khác. Đối chiếu độc lập toàn bộ 11 trace với PATH_TABLE xác nhận 11/11 trace khớp, bao phủ đủ 10 chuỗi khai báo trong lần chạy này.

5. Thảo luận và hạn chế

5.4. Đề xuất khắc phục lỗi và hoàn thiện bộ kiểm thử

Thứ nhất, kiểm tra đủ hai cận trước khi sao chép. Cần kiểm tra dấu trước khi đổi sang size_t; đồng thời kiểm tra kích thước nguồn và đích. Đoạn mã minh họa đề xuất:

```
int requested = tc->step5_audit_claim_len;
if (requested < 0) {
    /* Ghi nhan dau vao khong hop le. */
    return;
}
size_t length = (size_t)requested;
if (length > sizeof(old_bio_buf) || length > sizeof(diff_preview)) {
    /* Tu choi thay vi sao chep. */
    return;
}
memcpy(diff_preview, old_bio_buf, length);
```

Nếu dữ liệu đầu ra được sử dụng như chuỗi C, cần quy định thêm việc bảo đảm ký tự kết thúc. Kiểm tra kích thước sao chép chỉ xử lý an toàn vùng nhớ, chưa tự bảo đảm mọi yêu cầu ngữ nghĩa của chuỗi.

Thứ hai, xử lý kích thước bằng kiểu phù hợp. Nếu mục tiêu là tính số byte để cấp phát, nên dùng size_t và kiểm tra count > SIZE_MAX / sizeof(uint32_t) trước phép nhân. Nếu mô hình cố ý giới hạn kết quả ở 16 bit, cần kiểm tra so với UINT16_MAX trên giá trị đầy đủ trước khi ép kiểu. Thêm các giá trị biên như 16383, 16384 và 16385 vào bộ kiểm thử hồi quy.

Thứ ba, sửa cơ chế đánh giá kết quả. Tách nhãn tình huống mô phỏng khỏi lỗi thực; có thể dùng tên SIMULATED_RISK hoặc INVALID_INPUT. Chỉ thống kê crash thực khi tiến trình hoặc sanitizer cung cấp bằng chứng phù hợp. Sử dụng oracle độc lập để kiểm tra thuộc tính, thay vì chỉ dựa vào tên trạng thái được chính hàm mục tiêu ghi ra.

Thứ tư, chuẩn hóa đặc tả và số liệu. Điều chỉnh P03, P04, P07, P09, P10, P12 và P15 cho khớp yêu cầu thực sự; đồng thời cập nhật điều kiện trong code. Thay kích thước ghi cứng bằng thông tin đo được. Phần tổng kết mới đã dùng count_visited để tính các tử số; riêng tiêu đề bảng path còn ghi cố định “10/10” và nên chuyển thành giá trị tính toán. Bộ nhận dạng path cần so sánh cả độ dài lẫn toàn bộ dãy trạng thái, đồng thời phân biệt trace không thuộc đặc tả.

Thứ năm, kiểm tra quyền theo bản ghi. Với chức năng thành viên, truy vấn của người dùng thông thường cần gắn đồng thời id và clanId được suy ra từ ngữ cảnh xác thực. Không lấy phạm vi dòng họ do client tự khai báo làm căn cứ cấp quyền. Quyền quản trị liên dòng họ, nếu có, cần được định nghĩa riêng. Kiểm thử hồi quy phải có cặp tài khoản khác dòng họ và kiểm tra cả GET, PUT, DELETE.

Thứ sáu, sửa quy trình khởi tạo schema. Không loại cả câu SQL chỉ vì có chú thích mở đầu. Nên quản lý migration bằng các câu lệnh có cấu trúc hoặc công cụ migration, xử lý thứ tự tạo bảng và kiểm tra lại các cột mà route sử dụng. Chạy lại trên cơ sở dữ liệu trống để xác nhận quá trình cài đặt từ đầu, đồng thời kiểm tra không làm mất dữ liệu khi chạy lại trên cơ sở dữ liệu có sẵn.